University at Buffalo

Department of Computer Science and Engineering

CSE 573 – Computer Vision and Image Processing

Spring 2025

Name: Sumangowda Valagerepura Rajanna

UB No: 50606703

Title of the Project:

"Vision-Based Pedestrian Intention Prediction for Autonomous Navigation"

Project Overview

My project presents a pedestrian intent prediction model and the objective is to accurately detect pedestrians and estimate their short-term motion behavior using synthetic urban driving data generated from the CARLA simulator.

This project addresses a key limitation in state-of-the-art object detection models like YOLOv8, which often fail to detect small or partially occluded pedestrians, especially in visually complex environments. To overcome this, we incorporate semantic segmentation data provided by CARLA, enabling pixel-accurate localization of pedestrians that are missing in RGB frames alone. We then merge these two detection streams to ensure comprehensive pedestrian coverage.

Once all pedestrians are reliably detected, we compute dense optical flow between consecutive RGB frames to estimate motion within each pedestrian's bounding box. Based on the average flow magnitude, we classify each pedestrian as either "moving" or "stationary," thus predicting their immediate intent.

Requirements:

- RGB and semantic segmentation images from CARLA Simulator(Town1)
- YOLOv8-based bounding boxes
- Pixel-wise segmentation masks for pedestrians

Results:

- Merged pedestrian detections with spatial coverage improvements
- Motion-based intent labels (stationary or moving)
- Visual overlays and structured JSON metadata for each frame

Analysis:

- Designed a modular and accurate pedestrian detection system using enhanced image preprocessing, YOLOv8, and semantic segmentation.
- Implemented a bounding box merging algorithm to combine RGB and segmentation-based detections using IoU filtering.
- Developed a dense optical flow–based motion analysis algorithm to infer pedestrian intent.
- Visualized intent predictions per pedestrian and evaluated system accuracy across large-scale synthetic datasets.

The approaches lays a strong foundation for real-time pedestrian behavior modeling and can be extended to more complex applications such as trajectory forecasting, dynamic risk assessment, and multi-modal scene understanding in autonomous systems.

Experimental Protocol: Synthetic Data Collection

(Google Drive link: https://drive.google.com/drive/folders/1n6Au7ppnHXlYMKB-yLYRmIfrI8ptgLEh?usp=drive_link)

Overview

Our primary task in the data collection phase is to gather RGB images and semantic segmentation images from the CARLA simulator. The data collection phase is a very important part of my project, as this provides input data needed for the pedestrian intent prediction model. For this task, we collect RGB images and semantic segmentation (SS) images using the CARLA simulator. These images capture the interactions between pedestrians, vehicles, and the environment in the simulation.

The following parts are structure for collecting the necessary image data. The RGB images are used for visual perception, while the semantic segmentation (SS) images provide pixel-level labels to distinguish between different objects in the scene like pedestrians, vehicles, roads, trees and all the real-world entities.

Data Collection Process

The data collection process involves running the CARLA simulator to simulate the environment and collect images in real-time. Here's how it works:

- **Simulation Setup**:
  - The CARLA simulator is set to Town01, it is a predefined urban environment containing dynamic pedestrians and vehicles. Pedestrians interact with vehicles and the surrounding environment, which creates a realistic dataset for testing pedestrian intent models.
  - RGB Camera and SS Camera: These two cameras are mounted on a vehicle in the simulator. The RGB camera captures the visual scene, and the SS camera captures the semantic segmentation of the scene.
- **Data Collection**:
  - The RGB camera captures RGB images at a resolution of 1392x1024 pixels with a field of view (FoV) of 72 degrees, which simulates a real-time camera feed while the vehicle is moving in the Town 1 of the Simulator.
  - The semantic segmentation camera captures SS images at the same resolution and frame rate, providing pixel-wise class labels for various objects (pedestrians, vehicles, road surfaces).
- **Data Synchronization**:
  - The RGB and SS images are captured and synchronized in real-time where timestamps for both image types are aligned ensuring they correspond to the same moment in time.

Scripts for Data Collection

Detailed explanation of the scripts responsible for collecting the data, RGB and SS images.

➢ Main Script: KITTI_data_generator.py

This script is the primary script responsible for starting the simulation, setting up the sensors, and collecting the image data from the simulator.

- **Connects to the CARLA Server:**
  - The script connects to the CARLA simulator using the Python API and then initializes the client, connects to the CARLA server, and loads the Town01 map.
  - The CARLA server runs the simulation, and the script can now interact with it to start collecting data.

- **Spawns the Vehicle:**
  - It sets up a vehicle (Tesla Model 3) in Town01 using CARLA's spawn points. The vehicle is driven around the town during the data collection process, simulating a real-world autonomous driving scenario.
- **Camera Sensors:**
  - RGB Camera: Used to capture standard RGB images of the environment, providing visual data for pedestrian detection.
  - Semantic Segmentation Camera: Used to capture semantic segmentation images that classify each pixel in the image (road, vehicle, pedestrian).
  - These cameras are attached to the vehicle, ensuring they capture data from the vehicle's perspective.
- **Data Export:**
  - After collecting the images, the script saves them into the output folder (georef_colorized).
  - The images are saved with a timestamp and frame ID to ensure synchronization between the RGB and SS images.

➢ **generator_KITTI.py**

- This script plays a role in organizing and saving the collected data, ensuring it's structured in a way that can be easily used for further processing and testing.
- It will also be responsible for setting up the simulation and integrating other components like vehicle setup and camera sensor configuration.

➢ **ply.py**

- This script is primarily responsible for reading and writing PLY files, which are used for storing 3D point cloud data. Since we are focusing on RGB images and semantic segmentation images**.**

Data Collection Process Flow

Flow of how the scripts collectively collect data:

1. KITTI_data_generator.py starts the CARLA simulation, sets up the vehicle, and configures the RGB and SS cameras.
2. The simulation runs, and the cameras collect RGB and SS images in real-time at 10 Hz.
3. The images are saved in separate folders (images_rgb/ for RGB images and images_ss/ for SS images) with synchronized timestamps and frame IDs.
4. The collected data is saved to an output folder (georef_colorized/), making it ready for further processing or training the pedestrian intent prediction model.

Data Synchronization

The timestamps for both RGB images and SS images are synchronized. This is important because our pedestrian intent prediction model relies on matching RGB data with the corresponding semantic segmentation data, which is captured at the same time.

Each image file is saved with a frame ID and timestamp, ensuring that the images are correctly synchronized. The filenames like (0001_0.png) help in matching the RGB image with the SS image.

In the data collection phase, the CARLA simulator is used to collect RGB images and semantic segmentation images (SS). These images are captured using two camera sensors mounted on a vehicle, providing visual data that is essential for pedestrian detection and intent prediction. The data collector scripts are responsible for synchronizing

the image data, ensuring that both the RGB and SS images are saved with the correct timestamps for further processing.

Approach

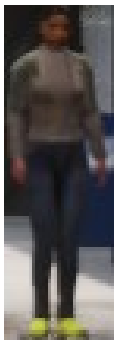
Part 1: Detecting Pedestrians on the RGB images

The first part is developing a pipeline for pedestrian detection on our CARLA Town01 synthetic data so I am using deep learning combined with traditional image processing that is enhanced YOLOv8 detection for the accurate pedestrian intention prediction.

- **Preprocessing image:**
  - **CLAHE (LAB Space):** Applied Contrast Limited Adaptive Histogram Equalization (CLAHE) on the luminance channel in LAB color space to enhance image contrast.
  - **Gamma Correction**: **Gamma correction (γ = 1.3**) implemented to improve visibility in darker regions.
  - **Sharpening**: Used a 3x3 sharpening kernel to highlight pedestrian edges.
  - **Bilateral Filtering:** Employed bilateral filtering to reduce noise while preserving edges and fine details.
- **Detection using YOLOv8:**

  Deployed YOLOv8 large model configured with:
  - **Confidence threshold: 0.25**
  - **IoU threshold: 0.45**
  - **Test-Time Augmentation (TTA): horizontal flips and scaling**

- **Post-processing:**
  - Bounding boxes annotated directly on original frames and cropped pedestrian images saved individually for detailed analysis. Structured JSON files generated containing bounding box coordinates and confidence scores.




```
{
  "frame": "0121_1",
  "pedestrians": [
    {
      "bbox": [
        209,
        500,
        235,
        573
      ],
      "confidence": 0.791
    }
  ],
  "count": 1
}
```


Performance Analysis:

- **Total Frames Processed**= 10,000
- **Total Pedestrians Detected**= 8,979 (12,310 total pedestrians)
- Achieved approximately **73%** detection accuracy.

Challenges:

- Difficulty in detecting the small and distant pedestrians.
- Misclassification around vertical high-contrast objects like dustbins and trees.

Tried with different approaches to increase the detection model with Histogram equalization in YUV space but it Increased visibility but elevated noise levels. Sobel edge overlays made Minimal detection improvements.

Morphological closing again Improved object silhouette but compromised detail. Lower confidence thresholds Increased recall but significantly elevated false positives.

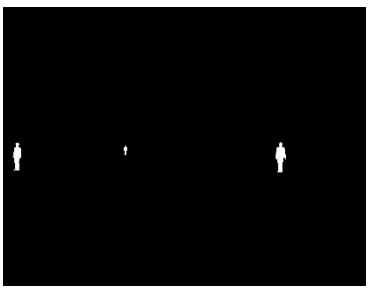
These methods provided localized improvements but did not surpass the initial pipeline's overall effectiveness.

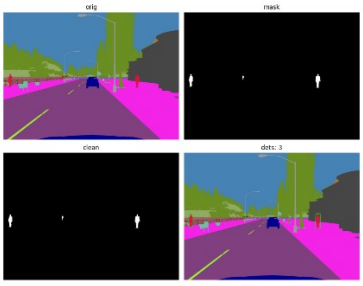
Part 2: Semantic Segmentation-Based Detection

The Semantic Segmentation detection (**detection_ss.py**) helps to enhance pedestrian detection performance by utilizing semantic segmentation data provided by the simulator. It specifically addresses the limitations of our RGB-based YOLO detection pipeline, specifically its reduced accuracy in detecting small pedestrians and those partially occluded by objects such as trees or poles—we incorporated semantic segmentation data provided by the simulator. This segmentation-based detection allows us to extract pedestrian regions with pixel-level precision, ensuring that individuals missing in the RGB frame due to visual clutter or low resolution are accurately captured. By recovering these missing detections, we enhance the continuity and completeness of pedestrian tracking across frames, which is critical for reliable and frame-consistent pedestrian motion and intent prediction.

Code Structure:

1. **Semantic Mask Extraction:**
    - The semantic segmentation frames are loaded and processed to isolate pedestrian pixels, identified by their unique RGB color.
    - A color-thresholding technique combined with a tolerance margin, and it ensures robust mask extraction even with slight color variations.
2. **Noise Reduction & Mask Refinement:**
    - Morphological operations, including closing (to fill gaps within detected pedestrian pixels) and opening (to remove isolated noise), are applied to refine the binary pedestrian mask.
    - A connected components analysis identifies contiguous pedestrian pixel regions, ensuring
3. **Bounding Box:**
    - Bounding boxes are created around each identified pedestrian component, based on the region's pixel dimensions and shape.
    - Criteria such as minimum area, height, and aspect ratio thresholds are employed to avoid false detections of non-pedestrian objects.
4. **Visualize of Output Generation:**
    - Bounding boxes are drawn directly onto the original semantic segmentation images.
    - Detailed metadata (bounding box coordinates, area, pedestrian count) is saved in structured JSON format.
    - A visualization frame with four panels, original semantic image, raw mask, cleaned mask, final detection result provides an immediate visual validation of the detection performance.





Explanation:

- **Semantic Image:**
  - Shows the original semantic segmentation provided by CARLA. Pedestrians appear in red, while other classes (vehicles, trees, poles, etc.) have distinct colors.
- **Binary Mask:**
  - Initially extracted pedestrian pixels appear white, clearly highlighting potential pedestrian regions, but also include noise.
- **Cleaned Binary Mask:**
  - Post-processing steps refine the mask to eliminate small, isolated noise, leaving clearly defined pedestrian areas.
- **Final Detections:**
  - The last panel demonstrates detected pedestrians marked with bounding boxes.
  - As per the provided JSON output, two pedestrians have been detected with bounding box coordinates clearly stated, reflecting precise detection locations and areas.

Contribution:

This semantic segmentation-based detection contributes to overcoming the limitations of the YOLO detection pipeline which fails to detect the low pixel level pedestrians. By accurately detecting pedestrians that YOLO may miss, particularly those far away or partially occluded, this approach provides complementary robustness for our project. Integrating these segmentation-based detections with YOLO outputs to substantially improve overall pedestrian detection accuracy, essential for the subsequent pedestrian intent prediction phase of the project.

Part 3: Merging YOLO and Semantic Segmentation Detections

By Merging, we obtain a complete and accurate set of pedestrian detections by combining outputs from both RGB-based YOLO detection and semantic segmentation-based detection pipelines. Although the YOLOv8 detector performs well under normal conditions, it frequently misses small or partially occluded pedestrians. Conversely, semantic segmentation can localize such pedestrians more reliably due to pixel-level class labeling. However, SS can sometimes produce noisy or merged outputs when pedestrians are close together. By merging both detection streams, we leverage their complementary strengths.

Methodology:

- Input Sources:
  - RGB frames from the Town01 dataset. YOLO-generated JSON files containing pedestrian bounding boxes. Semantic segmentation-based JSON files (post-processed) containing additional pedestrian bounding boxes.
- Frame Matching:
  - Each RGB frame has a YOLO JSON (**0000_0_detections. Json**) and a corresponding semantic segmentation detection frame (**0000_10_detections. Json**).
  - The suffix adjustment logic (+10) ensures correct alignment between RGB and SS annotations.
- Bounding Box Comparison and Filtering:
  - For each frame, the script compares bounding boxes from YOLO and SS detections.
  - **Intersection-over-Union (IoU)** is computed between SS and YOLO boxes.
  - Any SS box that does not sufficiently overlap (**IoU < 0.5**) with any YOLO detection is treated as a new, previously missed pedestrian.

Output Generated:

1. **Before Merging**

2. **After Merging**

- Merged visualizations are created:
  - YOLO detections are drawn in green.
  - Merged detections are drawn in red.
- In the initial YOLO-based detection (left image), only **two pedestrians** are correctly identified due to occlusion and distance constraints. However, after merging with the semantic segmentation output (right image), a **third pedestrian** previously missed is successfully detected and localized.
- Cropped images of SS-only detections are saved for review.
- A merged_summary.csv file records per-frame statistics: number of YOLO detections, number of SS-only additions, and total pedestrians after merging. (**0069_0,0,3,3**)

Contribution of Merging:

This merging strategy significantly improves pedestrian coverage, especially in cases where YOLO fails to detect individuals due to occlusion, distance, or scene clutter. By overlaying SS-only detections onto the original RGB frames, we ensure no pedestrian is left untracked across sequential frames.
This is particularly important for:

- Maintaining temporal consistency in pedestrian motion across frames.
- Providing a complete detection stream for the next stage—pedestrian intent prediction.
- Enhancing the overall accuracy and reliability of pedestrian understanding in complex urban scenes.

As a result, the merging pipeline forms a critical step in ensuring the completeness of our pedestrian behavior analysis system.

Final Part: Pedestrian Intent Prediction.

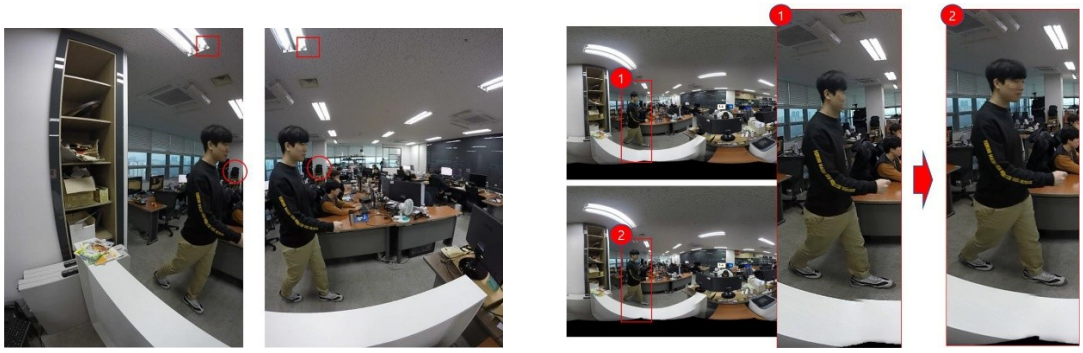
The primary goal of the intent prediction module is to estimate whether each detected pedestrian is likely stationary or moving, based on their observed motion between consecutive video frames. This temporal behavior analysis provides foundational insights for future tasks such as trajectory forecasting or collision risk estimation.

Methodology:

1. **ROI Generation:**
   - For each pair of sequential RGB frames (**frame_t and frame_t+1**), the corresponding merged pedestrian detections combining YOLO and semantic segmentation outputs are used as the region of interest (ROI).

- The RGB frames are **converted to grayscale** to reduce dimensionality and improve processing speed.

2. **Optical Flow Estimation:**
   - Dense optical flow is computed between the two grayscale frames using the **Farneback algorithm**, which estimates per-pixel motion vectors representing the apparent movement of objects.



   - This algorithm models the neighborhood of each pixel with polynomial expansion and finds the displacement that best matches local texture patterns between the two frames.

3. **Motion Vector Aggregation:**
   - For each detected pedestrian's bounding box, the optical flow vectors within that region are extracted.
   - The magnitude (speed) of each flow vector is computed, and a mean flow magnitude is calculated for the entire pedestrian ROI.

4. **Intent Classification Logic:**
   - A threshold-based decision rule is applied:
     - If the mean flow magnitude exceeds **0.5 pixels/frame**, the pedestrian is classified as **"moving."**
     - Otherwise, the pedestrian is considered **"stationary."**
   - This simple yet effective rule filters out background movement and camera jitter while focusing on actual pedestrian displacement.

5. **Output Representation:**

- Each pedestrian is assigned a motion state (moving or stationary) and the associated mean motion magnitude.
- The results are saved in JSON format and visualized by drawing bounding boxes:
    - Green for moving pedestrians
    - Red for stationary pedestrians
- Labels such as (moving 1.12px/f) are rendered under each pedestrian for clarity.

Performance and Outcome:

- The algorithm performs reliably across various scene contexts in Town01, even in cases where pedestrian motion is subtle or partially occluded.
- Since the prediction depends solely on the observed motion in pixel space, it remains interpretable and efficient.
- Combined with our robust detection from both RGB and semantic modalities, the system maintains temporal consistency in tracking pedestrian behavior.

Final Conclusion:

The intent prediction model adds a valuable behavioral layer to our detection pipeline. It ensures that each pedestrian is not only identified spatially but also interpreted temporally. This capability is essential for real-world autonomous systems where anticipating pedestrian actions is critical for planning, navigation, and safety assurance.

Finally, our project demonstrated the importance of combining multiple vision techniques to achieve reliable pedestrian intent prediction. While YOLOv8 provided a strong baseline, it struggled with small or occluded pedestrians. Integrating semantic segmentation significantly improved detection completeness. Also observed that accurate synchronization between RGB and segmentation frames was critical for successful merging and motion estimation.

Key Takeaways from The Project:

- Image preprocessing greatly improved detection quality.
- Semantic segmentation effectively complemented RGB-based detection.
- IoU-based merging ensured more complete pedestrian tracking.
- Optical flow provided a simple yet effective intent classification method.
- Proper frame alignment was essential for reliable results.

Reference Papers:

- Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). *CARLA: An Open Urban Driving Simulator.* In Proceedings of the 1st Annual Conference on Robot Learning (CoRL). CARLA Simulator
- Rasouli, A., Kotseruba, I., & Tsotsos, J. K. (2019). *Pedestrian Action and Intention Recognition: A Visual Perspective.* In Proceedings of the IEEE Conference on Intelligent Transportation Systems (ITSC). Benchmark for Evaluating Pedestrian Action Prediction | IEEE Conference Publication | IEEE Xplore
- CARLA-Kitti Setup & Installation Guide, fnozarian/CARLA-KITTI: CARLA-KITTI generates synthetic data from the CARLA simulator for KITTI 2D/3D Object Detection task.
- 2109.00892, NPM3D Benchmark Suite, jedeschaud/kitti_carla_simulator: KITTI-CARLA: Python scripts to generate the KITTI-CARLA dataset
- Ultralytics YOLOv8 – Ultralytics. *YOLOv8 Object Detection Models.* GitHub, 2023. https://github.com/ultralytics/ultralytics.git.
- Gunnar Farnebäck. *Two-Frame Motion Estimation Based on Polynomial Expansion.* In Proceedings of the 13th Scandinavian Conference on Image Analysis, 2003. OpenCV: Optical Flow
- Zhang, Y., Sun, P., Jiang, Y., Yu, H., & Jia, K. (2021). *Exploring CARLA for Semantic Segmentation: Benchmarks and Challenges.* In IEEE Intelligent Vehicles Symposium (IV).